

Fast Exponentiation, Floor/Ceil/Mod Tricks

A comprehensive technical reference covering algorithmic optimisation techniques in number theory and bit manipulation — from foundational concepts to cryptographic applications.

ALGORITHMIC OPTIMISATION

NUMBER THEORY

BIT MANIPULATION

Developed by Talenciaglobal

Topic Classification

Core Identity

Topic Name: Fast Exponentiation, Floor/Ceil/Mod Tricks

Category: Algorithmic Optimisation / Number Theory / Bit Manipulation

Domain: Computer Science → Algorithms → Mathematical Optimisation

Difficulty: Beginner to Intermediate (foundations); Advanced (modular tricks in cryptography)

Context & Relevance

Topic Type: Practical techniques with both conceptual and implementation depth

Prerequisites: Basic arithmetic, loops, binary representation, modulus operator

Industry Relevance: Critical for cryptography (RSA, ECC), hash tables, graphics (floor/ceil in pixel mapping), competitive programming, and database query optimisation

Recommended Learning Path



Naïve Exponentiation



Binary Exponentiation



Modular Exponentiation



Floor/Ceil Identities



Montgomery Reduction

What Is It? — Formal Definitions

Fast Exponentiation

Exponentiation by squaring — a method to compute **ab** (or **$ab \bmod m$**) in **$O(\log b)$** multiplications instead of $O(b)$, using the binary expansion of the exponent b .

Floor / Ceil Tricks

Identities and optimisations for integer division and rounding that avoid floating-point errors or performance pitfalls. Floor ($\lfloor x \rfloor$) rounds down; ceil ($\lceil x \rceil$) rounds up — both computable with pure integer arithmetic.

Mod Tricks

Properties of modulo arithmetic that simplify expressions, reduce overflow, and enable faster computation. Core identity: **$(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$** .

Simple Explanations

Fast Exponentiation

Instead of multiplying a by itself b times (slow), you square repeatedly. To compute 2^{13} , write 13 as binary **1101** ($8+4+1$). Compute $2^1, 2^2, 2^4, 2^8$ by squaring, then multiply only the powers corresponding to set bits.

Floor / Ceil

Floor ($\lfloor x \rfloor$) rounds down; ceil ($\lceil x \rceil$) rounds up. Tricks let you compute these without floating-point: floor division of positive integers is simply $a // b$. Ceil is $(a + b - 1) // b$.

Mod Tricks

Rules like $(a \times b) \bmod m = ((a \bmod m) \times (b \bmod m)) \bmod m$ keep numbers small and prevent overflow — essential when intermediate products would exceed 64-bit integer limits.

Main Components at a Glance

Technique	Core Operation	Key Identity
Fast Exponentiation	Repeated squaring	$a^b = \prod a^{(2^i)}$ for bits set
Floor Trick	Integer division	$\lfloor a/b \rfloor = a // b$ (positive)
Ceil Trick	Round-up division	$\lceil a/b \rceil = (a + b - 1) // b$ (positive)
Mod Trick	Reduction before op	$(a+b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$

Why Does It Exist? — Problems Solved

The Core Problems

1 Exponentiation at Scale

Naïve a^b needs b multiplications. For $b = 2^{30}$ (over 1 billion), this is computationally impossible in real time. RSA uses 2048-bit exponents — naïve methods would take **days** per decryption.

2 Float Precision Failures

Using float division and rounding functions is slow and introduces rounding errors for large integers. Integer-only floor/ceil avoids this entirely.

3 Integer Overflow

Computing $(a \times b) \% m$ directly can overflow the intermediate product when $a, b \approx 2^{31}$. Mod tricks keep values in range throughout computation.

What Happens Without These Techniques

- RSA decryption would take **days** instead of milliseconds
- Graphics loops would slow down using `math.floor` on every pixel
- Hash tables would require big-integer arithmetic for every operation
- TLS handshakes would be impractically slow
- Circular buffers would require expensive division on every access

 Industry stat: A modern web server handles ~10,000 TLS handshakes per second — only possible because modular exponentiation completes in ~1 ms per RSA-2048 operation.

Real-World Motivation



TLS Handshake

Modular exponentiation (RSA or Diffie-Hellman) happens in milliseconds during every HTTPS connection because of fast exponentiation. Without it, secure web browsing would be infeasible at scale.

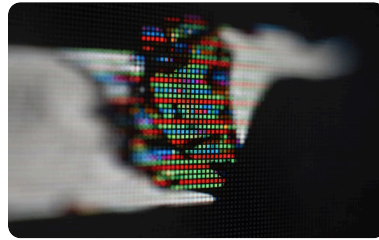


Image Scaling

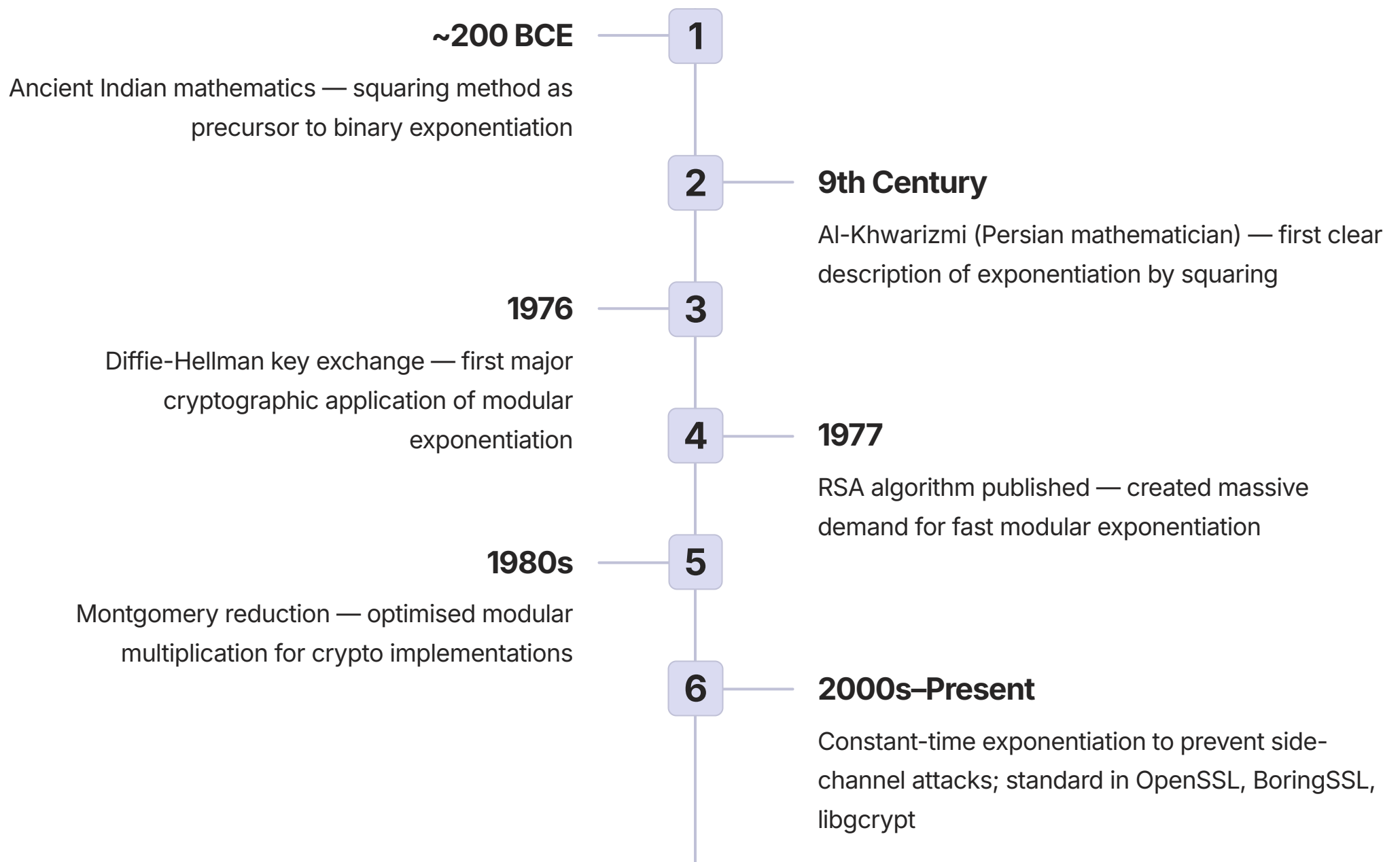
Computing source pixel index: $\text{floor}(x \times \text{src_width} / \text{dst_width})$ without floating point using integer math. Applied millions of times per frame in graphics pipelines.



Circular Buffer

$\text{index} = (\text{index} + 1) \% \text{size}$ — the modulo trick keeps the index within bounds efficiently. Used in audio buffers, network queues, and ring buffers across all operating systems.

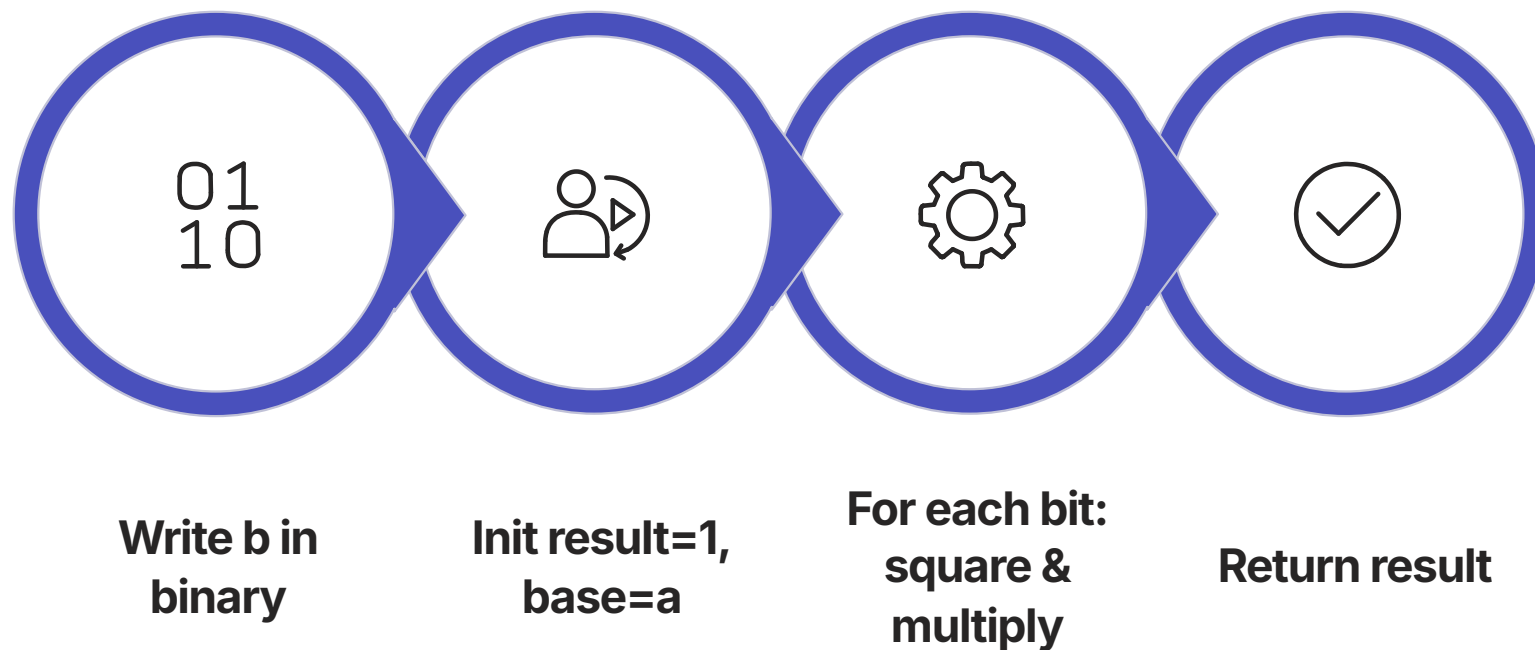
Evolution & History



i Future outlook: Post-quantum cryptography may use different algebraic structures, but exponentiation and modular reduction remain foundational to all current deployed cryptographic systems.

How It Works — Fast Exponentiation

Core mechanism: Represent exponent b in binary. For each bit, square the base; if the bit is 1, multiply into the result. This reduces $O(b)$ multiplications to $O(\log b)$.



This elegant mechanism transforms an exponential problem into a logarithmic one — the foundation of all modern public-key cryptography.

Worked Example: Computing 3^{13}

$b = 13 = \text{binary } 1101$ (bits from LSB: 1, 0, 1, 1)

Initialize: result = 1, base = 3

bit 0 = 1 → result *= base ($1 * 3 = 3$)

base = base² ($3^2 = 9$)

bit 1 = 0 → skip multiply

base = 9² (= 81)

bit 2 = 1 → result *= base ($3 * 81 = 243$)

base = 81² (= 6561)

bit 3 = 1 → result *= base ($243 * 6561 = 1,594,323$)

Result = $3^{13} = 1,594,323$ ✓ (only 4 multiplications vs 12 naïve)

How It Works — Floor, Ceil & Mod Tricks

Floor & Ceil Division

Floor Division

$\lfloor a/b \rfloor = a // b$ for positive integers. Integer division truncates toward zero (which equals floor for positives).

Ceil Division

$\lceil a/b \rceil = (a + b - 1) // b$ for $a, b > 0$. Proof: if $a = qb + r$ with $r > 0$, then $a + b - 1 = (q+1)b + (r-1)$, yielding quotient $q+1 = \text{ceil}$.

Safer Ceil (No Overflow)

$\lceil a/b \rceil = (a - 1) // b + 1$ for positive a . Avoids overflow when $a + b - 1$ might exceed 64-bit max.

Mod Trick Properties ($m > 0$)

- **Addition:** $(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$
- **Subtraction:** $(a - b) \bmod m = ((a \bmod m) - (b \bmod m) + m) \bmod m$
- **Multiplication:** $(a \times b) \bmod m = ((a \bmod m) \times (b \bmod m)) \bmod m$
- **Exponentiation:** $a^b \bmod m$ — use fast exponentiation with mod at each step
- **Negative numbers:** In C/Java, `%` gives remainder with sign of dividend. True modulus: $((a \% m) + m) \% m$

Types & Variants — Exponentiation Methods

Variant	Method	Complexity	Use Case
Naïve	b multiplications	$O(b)$	b very small (≤ 10)
Binary (right-to-left)	Square & multiply on bits	$O(\log b)$	General purpose
Binary (left-to-right)	Process bits MSB first	$O(\log b)$	Hardware implementations, constant-time crypto
k-ary exponentiation	Precompute powers $a^1..a^{(2^k-1)}$	$O(\log b)$ lower constant	Large exponents in crypto
Sliding window	Adaptive grouping of bits	$O(\log b)$	Optimised crypto (OpenSSL)

Trade-offs: Multiplications vs. Memory vs. Security

Method	Multiplications	Precomputation	Memory	Side-channel safe?
Naïve	b	None	$O(1)$	No
Binary (right-to-left)	$\sim 1.5 \log_2 b$	None	$O(1)$	Not constant-time
Binary (left-to-right)	$\sim 1.5 \log_2 b$	None	$O(1)$	Can be constant-time
Fixed-base window	$\log_2 b / k + 2^k$	2^k values	$O(2^k)$	No

Floor/Ceil & Mod Variants

Floor/Ceil Trick Variants

Problem	Formula (positive ints)	Example
Floor division	$a // b$	$7 // 3 = 2$
Ceil division	$(a + b - 1) // b$	$(7+3-1)//3 = 3$
Round to nearest (half up)	$(a + b//2) // b$	$(7+1)//3 = 2$
Integer square root floor	<code>isqrt()</code> (Python 3.8+)	<code>isqrt(10) = 3</code>

Mod Trick Variants

Category	Identity / Technique
Distributive over $+, -, \times$	Standard mod identities
Negative mod	$((x \% m) + m) \% m$
Power-of-two modulus	$x \& (m-1)$ when $m = 2^k$
Overflow-safe multiplication	$(a \% m) \times (b \% m) \% m$

✔ Industry stat: Bitwise AND for power-of-two modulus is up to **10x faster** than integer division for millions of hash table operations per second.

Real-World Use Case 1: RSA Decryption

Security / Web Servers

Business Problem: Decrypt incoming TLS messages by computing $c^d \bmod n$, where d and n are 2048-bit numbers. Without fast exponentiation, each decryption would take seconds — making HTTPS unusable at scale.

Solution Approach: Binary exponentiation combined with Montgomery multiplication for performance. Hardware acceleration (AES-NI) handles symmetric crypto; exponentiation runs in software or a dedicated crypto engine.

- ~1 ms per RSA-2048 decryption on a modern CPU
- A 2048-bit exponent requires ~3,000 modular multiplications
- OpenSSL uses sliding-window exponentiation for further optimisation

⚠ Security lesson: Also requires **blinding** — multiply by a random value before exponentiation — to prevent timing attacks that could reveal the private key.

Key Statistics

According to Netcraft, over **1.1 billion websites** use HTTPS. Each TLS handshake involves at least one modular exponentiation. Fast exponentiation is therefore executed billions of times per day globally.

A naïve implementation of RSA-2048 decryption (2048-bit exponent $\approx 2^{2048}$ multiplications) would take longer than the age of the universe on any hardware.

Real-World Use Case 2: Pagination & Use Case 3: Hash Tables

Pagination — Ceil Division

Industry: Web / E-commerce

Problem: Given total items (n) and items per page (k), compute number of pages without floating-point.

Solution: $\text{pages} = (n + k - 1) // k$

Benefit: One integer division instead of a ceil function call. Avoids float imprecision for large n .

Caution: For near-max n , use $(n - 1) // k + 1$ to avoid overflow in $n + k - 1$.

Hash Table Indexing — Mod Trick

Industry: Databases / Caching

Problem: Map arbitrary integer hash code to bucket index in range $[0, \text{size}-1]$.

Solution: $\text{hash} \& (\text{size} - 1)$ when size is a power of two.

Benefit: Bitwise AND is up to 10× faster than division/modulo for millions of operations.

Caution: Only works for powers of two; power-of-two sizes can weaken some hash distributions.

Circular Buffer — Mod Trick

Industry: Systems / Audio / Networking

Problem: Keep a buffer index within bounds $[0, \text{size}-1]$ on every increment.

Solution: $\text{index} = (\text{index} + 1) \% \text{size}$

Benefit: Single modulo operation replaces conditional branch. Used in every OS kernel ring buffer and audio driver worldwide.

Implementations — Fast Exponentiation

Example 1: Binary Exponentiation (Iterative, Python)

```
def fast_pow(a: int, b: int) -> int:
    """Return a^b using binary exponentiation. Works for non-negative b."""
    result = 1
    base = a
    exp = b
    while exp > 0:
        if exp & 1:      # if current bit is 1
            result *= base
        base *= base     # square the base
        exp >>= 1        # shift right (process next bit)
    return result

print(fast_pow(3, 13)) # 1594323
```

- ❗ This iterative right-to-left implementation uses $O(1)$ space and performs at most $2 \cdot \lfloor \log_2 b \rfloor$ multiplications — approximately $1.5 \cdot \log_2 b$ on average due to the expected number of set bits.

Example 2: Modular Exponentiation (Cryptography Style)

```
def mod_pow(base: int, exp: int, mod: int) -> int:
    """Compute (base^exp) % mod efficiently."""
    result = 1
    base %= mod # reduce base first to keep numbers small
    while exp > 0:
        if exp & 1:
            result = (result * base) % mod
        base = (base * base) % mod
        exp >>= 1
    return result

# Example: 5^117 mod 19
# By Fermat's little theorem: 5^18 ≡ 1 (mod 19), 117 mod 18 = 9
print(mod_pow(5, 117, 19)) # 1
```

Implementations — Ceil Division & Negative Mod

Example 3: Ceil Division and Negative Mod Wrap

```
def ceil_div(a: int, b: int) -> int:
    """Return ceil(a / b) for positive integers a, b."""
    return (a + b - 1) // b

def ceil_div_safe(a: int, b: int) -> int:
    """Overflow-safe ceil division for positive a, b."""
    return (a - 1) // b + 1

def true_mod(x: int, m: int) -> int:
    """Return x mod m in range [0, m-1] for any integer x, m > 0."""
    return ((x % m) + m) % m

# Examples
print(ceil_div(10, 3))    # 4
print(ceil_div(9, 3))    # 3
print(ceil_div_safe(10, 3)) # 4 (no overflow risk)
print(true_mod(-7, 5))    # 3 (because  $-7 \equiv 3 \pmod{5}$ )
print(true_mod(-1, 5))    # 4
print(true_mod(12, 5))    # 2
```

✓ **Python note:** Python's built-in `pow(base, exp, mod)` is implemented in C, highly optimised, and generally constant-time on modern platforms. Always prefer it over a custom implementation for production cryptographic code.

⚠ **C/Java note:** The `%` operator returns a remainder with the sign of the dividend. `-7 % 5` returns `-2` in C, not `3`. Always apply the `true_mod` wrapper when working with negative values.

Hands-On Lab 1: Naïve vs. Fast Exponentiation

01

Write Naïve Implementation

Implement `pow_naive(a, b)` using a simple loop: multiply `a` by itself `b` times. This is $O(b)$.

02

Implement Fast Version

Use the `fast_pow(a, b)` binary exponentiation from the previous section. This is $O(\log b)$.

03

Benchmark Both

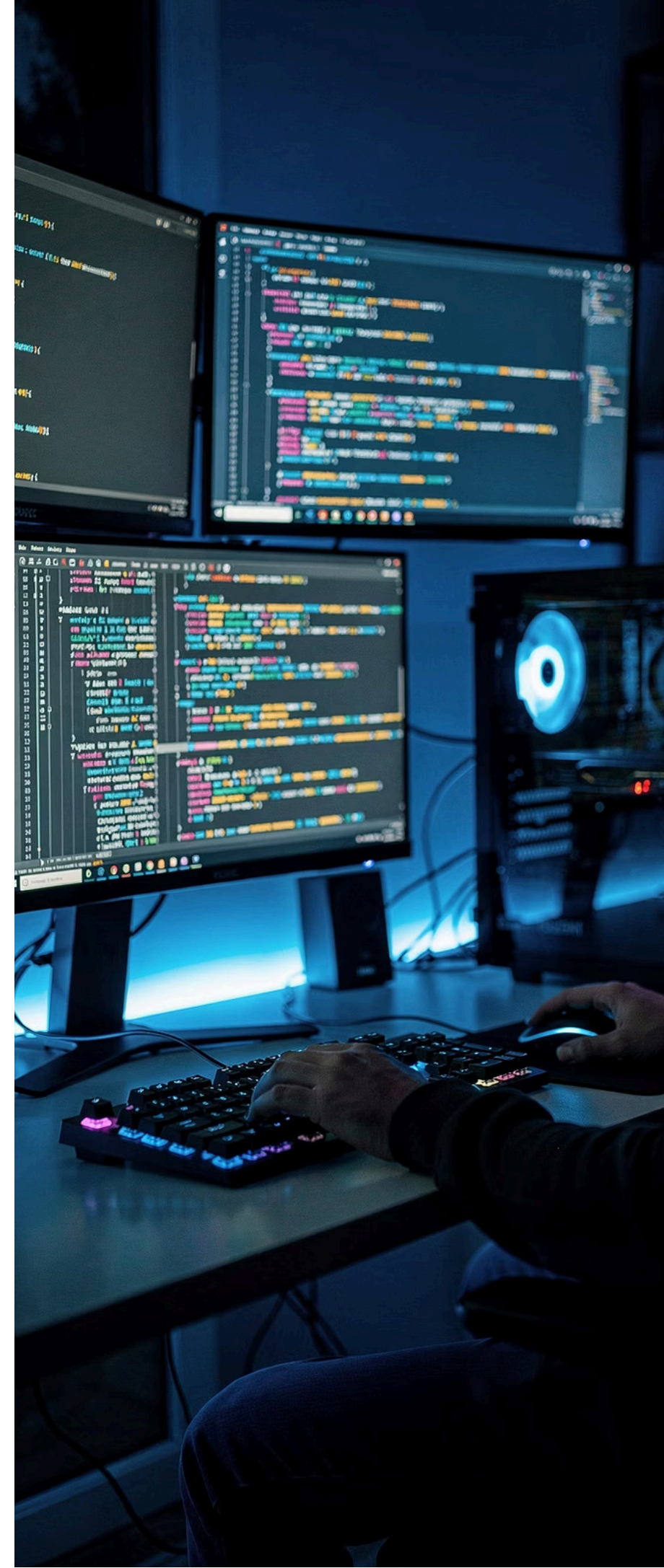
Compute `3 ** 1000000` (large exponent) using both. Time each using `time.perf_counter()`.

04

Observe & Validate

Naïve will be extremely slow (or hang) for $b = 10^6$; fast exponentiation returns instantly. Verify both produce identical results for small b .

- ✔ **Learning outcome:** Understand why logarithmic complexity is essential for large exponents. For $b = 2^{30} \approx 10^9$, binary exponentiation uses only ~ 30 multiplications vs. 1 billion naïve multiplications.



Hands-On Lab 2: Mod Tricks — Overflow Prevention

Objective: Simulate 32-bit overflow and demonstrate how mod tricks prevent it

Instructions

Python integers don't overflow natively, but simulate 32-bit behaviour using `& 0xFFFFFFFF`.

```
# Simulate 32-bit integers
a = 0xFFFF0000 # ~4.29 billion
b = 0xFFFF0001
m = 0xFFFFFFFF # 2^32 - 1

# Method 1: Direct (overflows in real 32-bit C)
direct = (a * b) % m

# Method 2: Mod trick (keeps intermediates small)
safe = ((a % m) * (b % m)) % m

print(direct == safe) # True — same result
print(a * b)         # Huge intermediate in Python
print((a % m) * (b % m)) # Much smaller intermediate
```

Expected Outcomes

- Both methods yield the same final result
- Method 2 keeps intermediate values $< m^2$
- In real 32-bit C, Method 1 would overflow silently — a critical bug

Learning Outcome

Mod tricks prevent overflow in languages with fixed-width integers. This is why every cryptographic library applies modular reduction at each multiplication step, not just at the end.

-  Real-world impact: Silent integer overflow in cryptographic code has caused multiple CVEs (Common Vulnerabilities and Exposures) in production systems.

Advantages & Disadvantages

Fast Exponentiation

✓ Advantages

- Logarithmic time: $O(\log b)$ vs $O(b)$
- Memory efficient: $O(1)$ space
- Essential for all public-key cryptography
- Composable with modular reduction

✗ Disadvantages

- Slightly more complex code than naïve loop
- Constant-time variant adds overhead

Floor/Ceil Tricks

✓ Advantages

- Avoids floating-point entirely
- Faster than `math.floor` (no type conversion)
- Exact for all integer inputs

✗ Disadvantages

- Ceil formula $(a+b-1)$ may overflow for near-max values
- Behaviour differs for negative numbers across languages

Mod Tricks

✓ Advantages

- Prevents overflow in fixed-width arithmetic
- Keeps numbers small throughout computation
- Bitwise AND variant is extremely fast

✗ Disadvantages

- Negative modulo behaviour not always obvious
- Power-of-two trick only applies to specific moduli

Performance Considerations

Complexity Analysis

Operation	Time	Space
Naïve exponentiation	$O(b)$	$O(1)$
Binary exponentiation	$O(\log b)$ mults	$O(1)$
Ceil division (positive)	$O(1)$ integer ops	$O(1)$
Mod multiplication reduction	$O(1)$	$O(1)$

Key Metrics

- **Exponentiation:** Multiplications = $\lfloor \log_2 b \rfloor + \text{popcount}(b) - 1$. For a 1024-bit exponent: ~1,536 multiplications.
- **Floor/ceil:** Latency = 1–3 CPU cycles for integer division (pipelined).
- **Mod:** % is slower than & for power-of-two modulus by ~2x.

Common Bottlenecks

→ Big-Integer Multiplication

For large moduli (RSA-2048), each multiplication is itself expensive. Use Montgomery reduction to avoid full division at each step.

→ Ceil Overflow

$a + b - 1$ may overflow 64-bit for near-max values. Use $(a - 1) // b + 1$ as the safer alternative.

→ Float Conversion

`math.floor` calls the C library and involves type conversion. Prefer `//` for integer floor in performance-critical loops.

Scalability & Reliability

Technique	Small Input	Large Input	Massive (Crypto)
Fast exponentiation	Naïve fine ($b \leq 10$)	Binary exponentiation	Binary + Montgomery reduction
Ceil division	Direct formula	Same formula	Same (no change needed)
Mod tricks	Use %	Use % with reduction	Use & for power-of-two

Scaling in Practice

Small Scale

Exponent $b = 10 \rightarrow$ naïve loop may be faster due to overhead of bit-loop setup. Binary exponentiation overhead is negligible but measurable.

Medium Scale

$b = 10^6 \rightarrow$ binary exponentiation uses ~ 20 multiplications; naïve uses 1 million. Difference: milliseconds vs. minutes.

Enterprise Scale

RSA with 4096-bit exponent ($\sim 1.2 \times 10^3$ bits) $\rightarrow$ binary exponentiation uses $\sim 6,000$ multiplications; still sub-millisecond with hardware acceleration.

Design & Architectural Considerations

Key Design Decisions

When to Use Fast Exponentiation

Any exponent $> 1,000$, or any modular exponent. For tiny exponents, naïve may be simpler and faster due to loop overhead.

Left-to-Right vs. Right-to-Left

Left-to-right allows constant-time implementation (critical for crypto). Right-to-left is simpler to implement but branches on each bit.

Power-of-Two Modulus

Use bitwise AND for speed, but ensure hash distribution is adequate. Power-of-two sizes can amplify weaknesses in some hash functions.

Design Patterns



Strategy Pattern

Interchangeable exponentiation algorithms selected based on exponent size and security requirements.



Decorator Pattern

Add blinding (multiply by random factor) to modular exponentiation as a security decorator without changing core logic.



Template Method

Exponentiation skeleton with pluggable multiplication routine — swap Montgomery vs. naïve multiplication without changing the bit-loop.

i Maintainability: Use Python's built-in `pow(base, exp, mod)` for modular exponentiation — it is highly optimised in C and avoids re-implementation risk. Only write custom implementations when platform constraints require it.

Security Considerations

❌ Fast exponentiation is a critical security primitive. Incorrect implementation can expose private cryptographic keys through timing attacks, power analysis, or fault injection.

Timing Attacks

Exponentiation that takes different time for different bits leaks the private exponent. **Mitigation:** Use constant-time square-and-multiply — always square, always multiply (using conditional move, not branch), with identical memory access patterns.

Power Analysis

Physical measurement of power consumption during exponentiation can reveal key bits. **Mitigation:** Apply blinding — multiply base by a random value r before exponentiation, divide result by r^{exp} afterward.

Fault Injection

Inducing hardware faults during computation can produce incorrect results that leak key material. **Mitigation:** Verify results before returning; use redundant computation.

Hardening Checklist

- Always use `pow(base, exp, mod)` with three arguments in Python — constant-time on most platforms
- Never write custom crypto exponentiation without blinding and constant-time guarantees
- Use `cryptography.hazmat.primitives` for production cryptographic operations
- FIPS 140-2 compliance requires validated implementations — use OpenSSL FIPS module

Best Practices

✓ Do's

Area	Recommendation
Design	Use built-in pow for modular exponentiation in high-level languages
Development	Implement ceil division as $(a + b - 1) // b$ only when a, b positive and no overflow risk
Security	For crypto, use constant-time exponentiation from <code>cryptography.hazmat.primitives</code>
Performance	Use $x \& (size-1)$ for modulo by power-of-two
Operations	Prefer integer arithmetic over floating-point for floor/ceil in performance loops

× Don'ts

Area	Avoid
Design	Don't use naïve exponentiation for large exponents ($b > 1,000$)
Development	Don't assume % on negative numbers gives modulus — use <code>true_mod</code> wrapper
Security	Don't implement RSA exponentiation yourself without blinding
Performance	Don't use <code>math.floor</code> when <code>//</code> works
Operations	Don't ignore overflow in $a + b - 1$ for ceil division with large numbers

Common Mistakes & Pitfalls

Beginner Level

Mistake	Why It Happens	Impact	Correct Approach
Using loop exponentiation for large exponent	Unaware of fast method	Program hangs	Use binary exponentiation or pow()
Assuming % on negatives gives positive result	C/Java background	Off-by-one errors	Use ((x % m) + m) % m
Off-by-one in ceil division	Forgetting formula	Wrong page count	Use (a + b - 1) // b or (a - 1) // b + 1

Intermediate Level

Mistake	Why It Happens	Impact	Correct Approach
Right-to-left exponentiation in crypto without blinding	Security unaware	Timing leakage	Add random blinding factor
Overflow in ceil formula when a, b near max	$a+b-1$ exceeds $2^{63}-1$	Wraparound in C	Use $(a-1)//b + 1$ for positive a
Not reducing base before modular exponentiation	Intermediate big numbers	Slow, memory heavy	Apply base %= mod at start

Advanced / Production Level

Mistake	Why It Happens	Impact	Correct Approach
Re-computing exponentiation for same base and modulus	No caching	Wasted computation	Use fixed-base precomputation
Non-constant-time exponentiation in signature verification	Copy-paste from textbook	Timing attack	Use library constant-time variant
Assuming pow(base, exp, mod) is always constant-time	Python docs unclear	Potentially vulnerable	Check documentation; use dedicated crypto lib

Comparison with Alternatives

Exponentiation Methods

Method	Multiplications	Constant-time?	Use Case
Naïve	b	Yes	$b \leq 10$
Binary (right-to-left)	$\sim 1.5 \cdot \log_2 b$	No (branch on bit)	General non-crypto
Binary (left-to-right)	$\sim 1.5 \cdot \log_2 b$	Yes (conditional move)	Crypto
k-ary (k=4)	$\sim \log_2 b + 2^k$	No	Optimised big int
Sliding window	$\sim \log_2 b + \text{small}$	No	OpenSSL

Floor/Ceil Alternatives

Approach	Speed	Precision
Integer floor division (//)	Fast	Exact
math.floor(a / b)	Slower (float)	May round incorrectly for large ints
int(a / b) truncation	Fast	Wrong for negatives

Decision Tree: Which Exponentiation?

Need modular exponent?
├─ Yes → Use **pow**(base, exp, mod)
│ (C-optimised, often constant-time)
└─ No → Is exponent large (> 1000)?
 ├─ Yes → Use binary exponentiation
 │ (fast_pow)
 └─ No → Naïve loop is fine

Mod Trick Alternatives

Approach	Speed	Applicability
Direct %	Baseline	All moduli
$\& (m-1)$ for 2^k	$\sim 2\times$ faster	Only $m = 2^k$
Lookup table (small m)	Very fast	m small, precomputed

Learning Summary & Key Takeaways



Fast Exponentiation

Reduces exponentiation from $O(b)$ to $O(\log b)$ by repeated squaring. Essential for RSA and any large exponent. For $b = 2^{1024}$, this is the difference between ~1,500 multiplications and 10^{308} multiplications.



Ceil Division

Formula $(a + b - 1) // b$ avoids floating-point entirely. Use safer form $(a - 1) // b + 1$ when overflow is a risk. One integer operation replaces a float conversion and rounding call.



Mod Tricks

Keep numbers bounded and enable distribution:
 $(a \times b) \% m = ((a \% m) \times (b \% m)) \% m$. Negative modulo requires $((x \% m) + m) \% m$ for true range $[0, m-1]$.



Bitwise Optimisation

Bitwise AND for mod by power-of-two is ~2× faster than division. Precompute when base and modulus are fixed. Prefer `//` over `math.floor` in performance loops.

Revision Cheat Sheet

Quick Reference Formulas

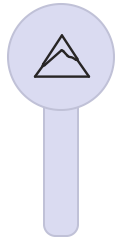
Operation	Formula / Code
Binary exponentiation	<code>while b: if b&1: r*=a; a*=a; b>>=1</code>
Ceil(a/b)	<code>(a + b - 1) // b</code> (watch overflow)
Safer ceil	<code>(a - 1) // b + 1</code>
True modulus (negative)	<code>((x % m) + m) % m</code>
Mod by power-of-two	<code>x & (m-1)</code> where $m = 2^k$
Modular exponentiation	<code>pow(base, exp, mod)</code>

Memory Aid: "Square and multiply, bits decide; ceil add divisor minus one divide."

Common Interview Questions

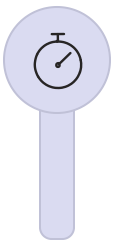
- **Implement `pow(a, b, mod)` without built-in.** → Binary exponentiation with modulo at each step.
- **How many multiplications for 2^{100} ?** → $\log_2(100) \approx 7$ squarings + `popcount(100)` = 3 multiplies → ~10 total.
- **Compute `ceil(17/5)` using integer arithmetic.** → $(17+5-1)//5 = 21//5 = 4$.
- **What is `(-7) % 5` in Python vs C?** → Python: 3 (non-negative). C: -2 (sign of dividend).
- **Why does `x & (size-1)` work for modulo?** → $\text{size} = 2^k$, so binary mask captures the lower k bits exactly.
- **Explain timing attack on RSA.** → Bits of exponent affect whether multiply occurs, revealing key through execution time differences.

Recommended Next Topics



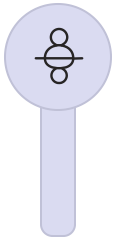
Montgomery Reduction

Faster modular multiplication for repeated operations — the backbone of high-performance RSA and ECC implementations in OpenSSL and hardware crypto engines.



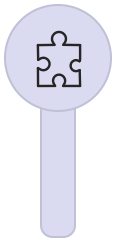
Constant-Time Cryptography

Writing side-channel resistant exponentiation — techniques including conditional moves, blinding, and memory access pattern normalisation to prevent timing and power attacks.



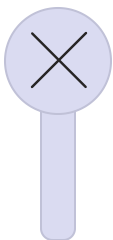
Modular Inverse

Using the extended Euclidean algorithm with mod tricks to compute $a^{-1} \bmod m$ — essential for RSA key generation and elliptic curve arithmetic.



Chinese Remainder Theorem

Faster RSA decryption by splitting computation across smaller moduli using mod tricks — reduces decryption time by $\sim 4\times$ in practice.



Bit Twiddling Hacks

Advanced integer optimisations — computing popcount, isolating lowest set bit, reversing bits, and other techniques that underpin high-performance systems programming.